

Project 5: Building a Retail Data Pipeline

Literature Review

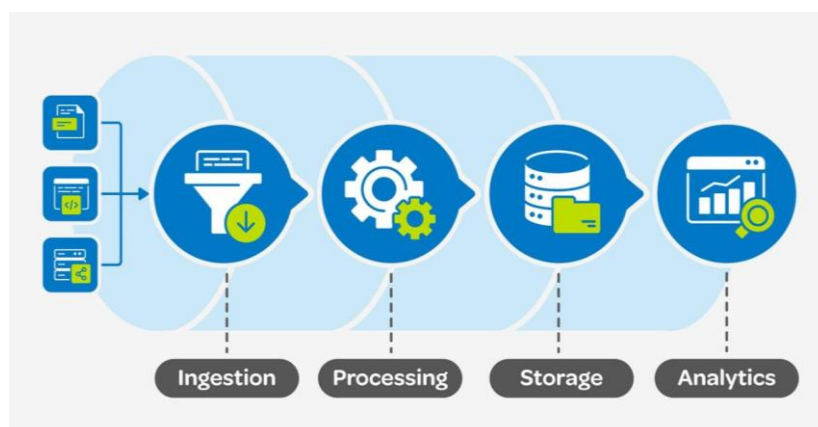
1. Introduction

Retail businesses generate massive amounts of data from daily operations such as sales transactions, customer interactions, and inventory management. In the modern digital era, this data has become a critical asset that organizations rely on to improve their performance and gain a competitive advantage.

Data-driven decision making refers to the process of using collected data and analytical techniques to guide business decisions instead of relying on intuition or assumptions. By analyzing patterns, trends, and customer behavior, organizations can make more accurate and strategic decisions.

In the retail domain, data analytics plays a vital role in optimizing key business operations such as inventory management, pricing strategies, and customer experience. For example, analyzing sales data helps businesses forecast demand and improve overall supply-chain efficiency.

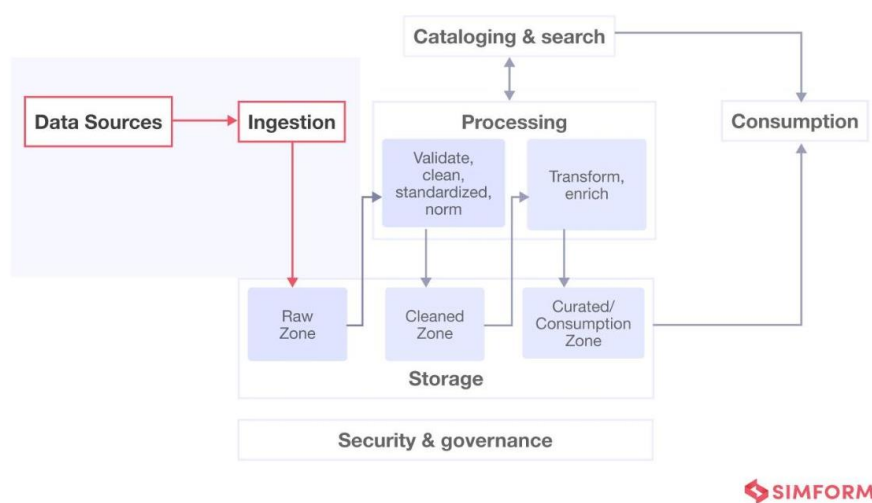
Data engineering is a fundamental enabler of data-driven decision making. It is responsible for collecting, cleaning, and transforming raw data into structured formats that can be analyzed effectively. A well-designed data pipeline ensures that data flows reliably from source systems to analytical stores with minimal latency and maximum accuracy.



2. Existing Systems and Related Work

Existing systems for handling retail data rely on data engineering architectures that support efficient data collection, processing, and analysis. These systems manage large volumes of data from multiple sources such as point-of-sale transactions, customer data, and inventory management systems.

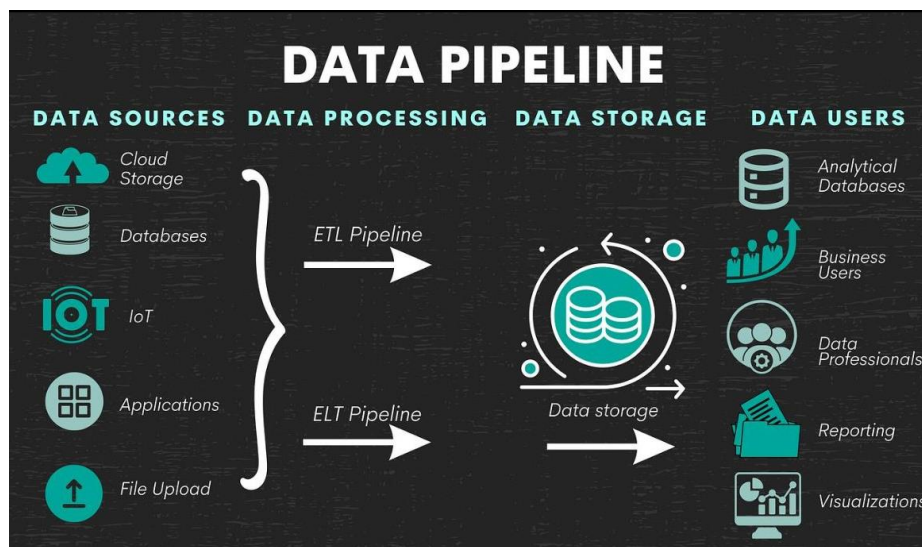
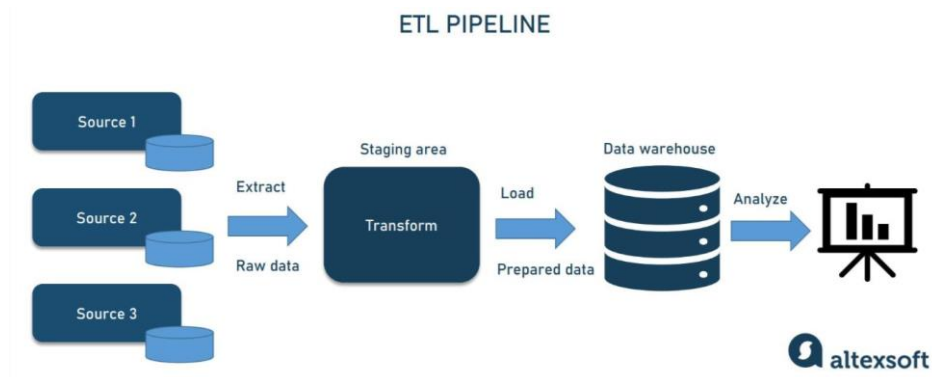
A key component in these systems is the data pipeline, which moves data from raw sources to storage and analysis layers. It typically includes stages such as data ingestion, data transformation, and data storage, as well as cross-cutting concerns like cataloging, security, and governance.



2.1 ETL and ELT Architectures

The ETL (Extract, Transform, Load) process is the traditional approach, where data is extracted from source systems, transformed in a staging area, and then loaded into a data warehouse. This approach gives fine-grained control over data quality before loading.

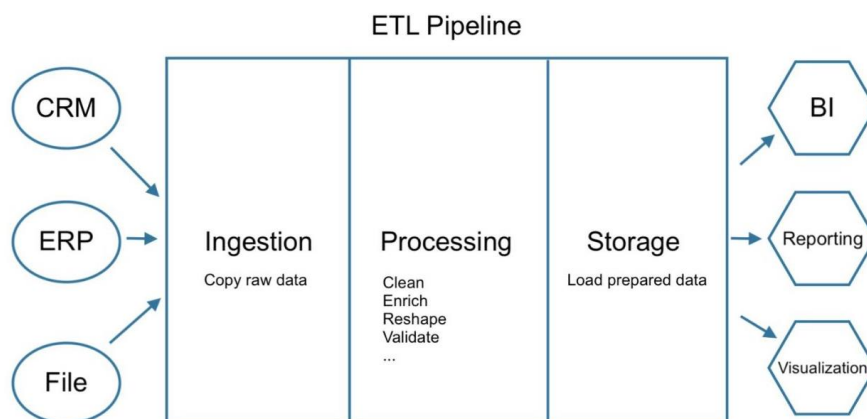
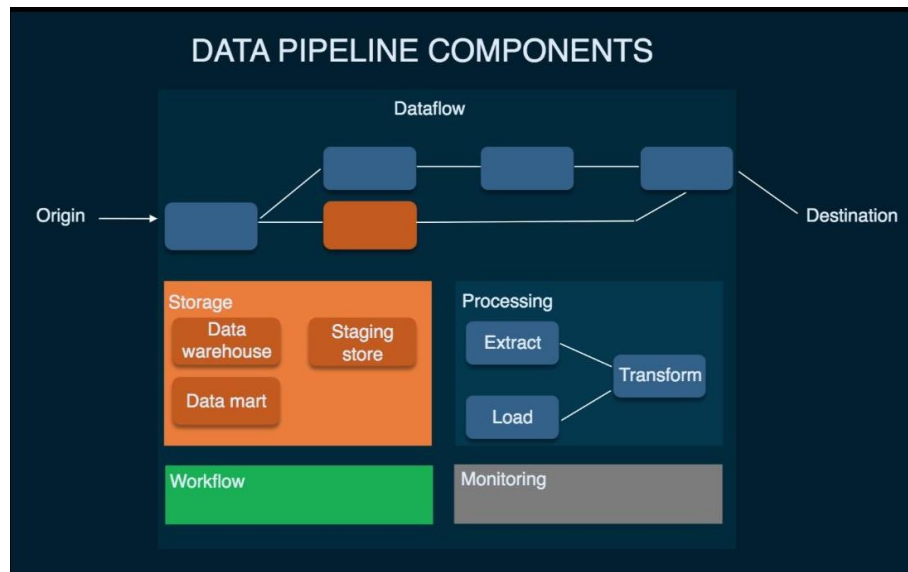
In modern cloud-native systems, ELT (Extract, Load, Transform) is increasingly preferred — raw data is loaded first into a cloud warehouse such as BigQuery or Snowflake, and transformations are performed using in-warehouse SQL tools such as dbt. This improves scalability and leverages the elastic compute power of cloud platforms.



2.2 Data Pipeline Components

A modern data pipeline consists of several interdependent components. Figure 5 below illustrates the core building blocks: dataflow management, storage layers (data warehouse, staging store, data mart), ETL processing sub-tasks (Extract, Transform, Load), workflow orchestration, and monitoring.

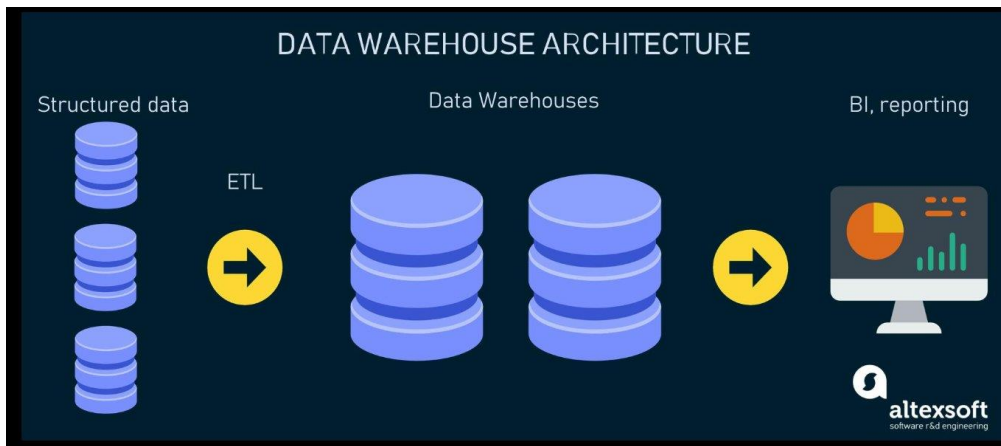
- **Ingestion:** The process of copying raw data from heterogeneous sources (CRM, ERP, flat files) into a staging layer without modification.
- **Processing:** Cleaning, enriching, reshaping, and validating data to ensure it meets analytical quality standards.
- **Storage:** Loading prepared data into structured stores — a Data Warehouse for enterprise queries and Data Marts for domain-specific reporting.
- **Workflow / Orchestration:** Scheduling and coordinating pipeline tasks using tools like Apache Airflow.
- **Monitoring:** Tracking job success rates, data freshness, and alerting on failures.



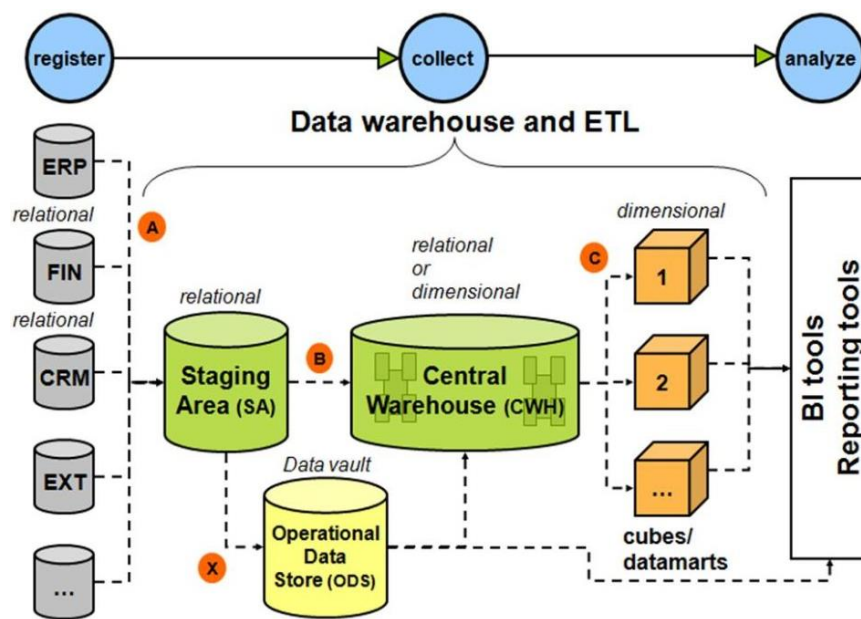
2.3 Data Warehouse Architectures

Data warehouses serve as centralized storage systems that allow efficient querying and analysis of large datasets, supporting business intelligence operations. Three common architectural patterns are used in practice.

The classic data warehouse architecture moves structured data through ETL into one or more warehouse nodes, then serves BI and reporting dashboards. This is the most widely deployed pattern in enterprise retail systems.

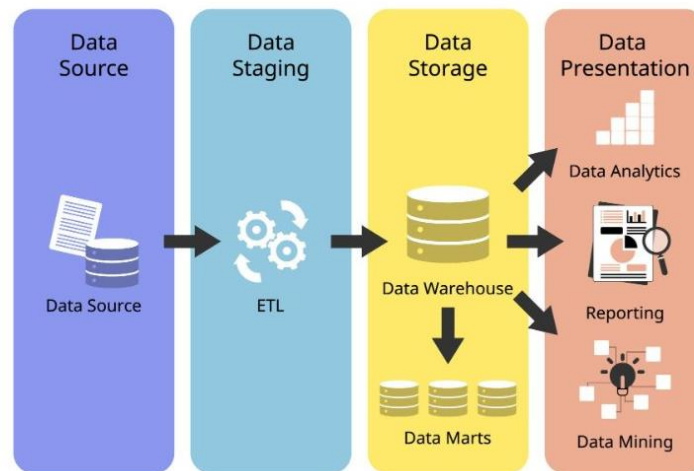


A more detailed enterprise architecture introduces a Staging Area (SA), a Central Warehouse (CWH), an Operational Data Store (ODS), and dimensional cubes / data marts at the consumption layer. This layered approach (Figure 8) is used by large retailers to support both operational and analytical workloads.



The four-layer warehouse pattern (Data Source → Data Staging → Data Storage → Data Presentation) provides a clear separation of concerns: raw ingestion, ETL transformation, warehouse / data mart storage, and finally analytics, reporting, and data mining.

Data Warehouse Architecture



2.4 Modern Data Architecture and Data Lakes

In the modern data stack, a Data Lake acts as a central raw-data repository that feeds multiple downstream systems: relational databases, non-relational stores, data warehouses, machine learning platforms, and log analytics pipelines. This architecture, shown in Figure 10, is becoming standard in cloud-native retail deployments.

Modern Data Architecture

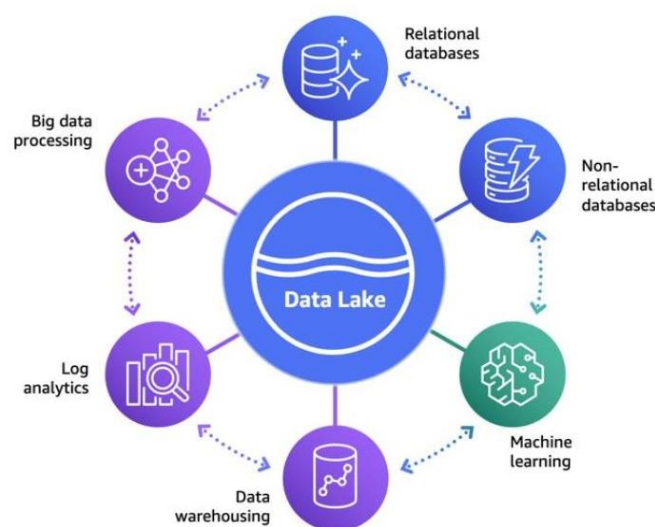
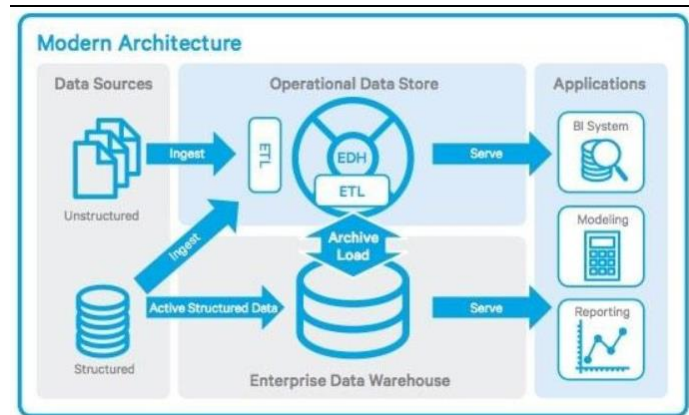
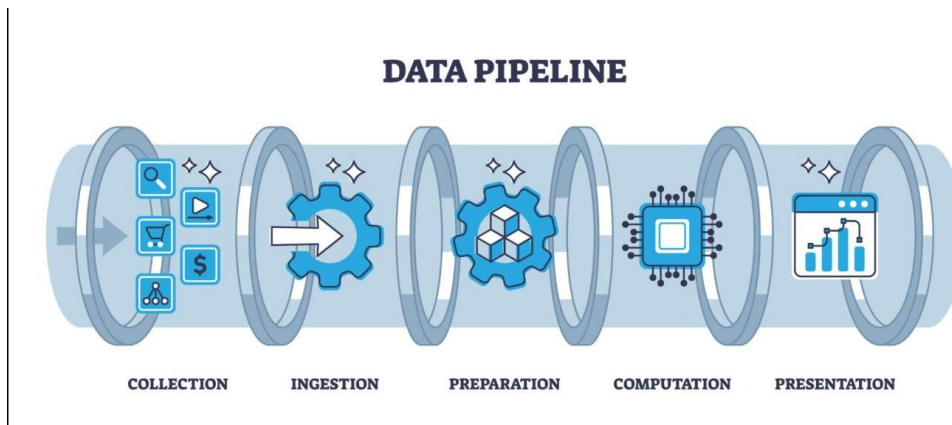


Figure 11 shows a modern operational architecture where both unstructured and structured data sources are ingested via ETL into an Enterprise Data Hub (EDH) and an Enterprise Data Warehouse, then served to BI systems, modeling tools, and reporting applications.



2.5 Pipeline Stages in Detail

Figure 12 provides a granular view of the five-stage data pipeline: Collection → Ingestion → Preparation → Computation → Presentation. This model aligns directly with the retail pipeline we are building, where retail CSV/SQL data is collected, ingested, cleaned, analytically processed, and finally presented via dashboards.



3. Limitations of Existing Approaches

Despite the advancements in data engineering systems, several challenges still exist that informed the design decisions for this project.

- Pipeline complexity: Managing multi-source pipelines with heterogeneous formats (SQL + Parquet + CSV) requires careful schema management and error handling at every stage.

-
- Data quality issues: Missing, duplicate, or inconsistent records can propagate through the pipeline and corrupt downstream analytics. Continuous validation and monitoring are required.
 - Scalability constraints: Traditional on-premise ETL systems struggle with increasing data volumes, leading to performance bottlenecks and higher infrastructure costs.
 - Real-time limitations: Conventional batch data warehouses are not optimized for low-latency streaming scenarios, which limits responsiveness for time-sensitive retail decisions.
 - Cost of cloud warehouses: Services like BigQuery and Snowflake, while powerful, can incur significant costs at scale if queries are not optimized and data is not partitioned correctly.
 - Operational overhead: Maintaining orchestration tools like Airflow requires DevOps expertise that may not always be available in smaller teams.

4. Proposed Solution

To overcome these limitations, this project proposes building a scalable retail data pipeline that efficiently processes Walmart-style sales data and transforms it into actionable business insights.

The system follows a structured pipeline consisting of data ingestion from SQL and Parquet sources, data transformation using Python (Pandas/Polars) for cleaning, deduplication, and enrichment, and data storage in an analytics-ready format (Parquet → BigQuery). Workflow orchestration is handled by Apache Airflow running in a Docker Compose environment.

By applying hybrid ETL/ELT processes and leveraging optimized columnar storage (Parquet), the system ensures better performance, scalability, and data reliability compared to traditional flat-file approaches. The modular pipeline design also makes it straightforward to extend with additional data sources or real-time streaming in future iterations.

This approach directly addresses the common challenges identified in Section 3: data quality is enforced at the ingestion stage, scalability is achieved through columnar storage and cloud warehousing, and operational complexity is managed through containerization and automated scheduling.

5. References

The following sources informed the research and design decisions in this document:

1. IBM Think — Data Pipeline Types and Architecture. <https://www.ibm.com/think/topics/data-pipeline-types>
 2. GeeksforGeeks — ETL (Extract, Transform, Load) Process in Data Engineering. <https://www.geeksforgeeks.org/data-engineering/explain-the-etl-extract-transform-load-process-in-data-engineering/>
-

-
3. GeeksforGeeks — 10 Best Data Engineering Tools for Big Data Processing.
<https://www.geeksforgeeks.org/data-engineering/10-best-data-engineering-tools-for-big-data-processing/>
 4. LatentView Analytics — Data Engineering in Retail. <https://www.latentview.com/blog/data-engineering-in-retail/>
 5. ThoughtSpot — Data Warehouse Architecture. <https://www.thoughtspot.com/data-trends/data-modeling/data-warehouse-architecture>
 6. AltexSoft — ETL Process and Data Warehouse Architecture. <https://www.altexsoft.com/blog/etl-process/>
 7. Medium / Mayura Rangdal — Understanding ETL and ELT for Data Pipelines.
<https://medium.com/@mayura.rangdal/understanding-etl-and-elt-for-data-pipelines>
-